

SA-MIRI 2025

Practice Pi: Parallelizing Model Training with Distributed Data Parallel (with PyTorch)

Jakub Seliga (jakub.seliga@estudiantat.upc.edu)

Thomas Aubertier (thomas.aubertier@estudiantat.upc.edu)

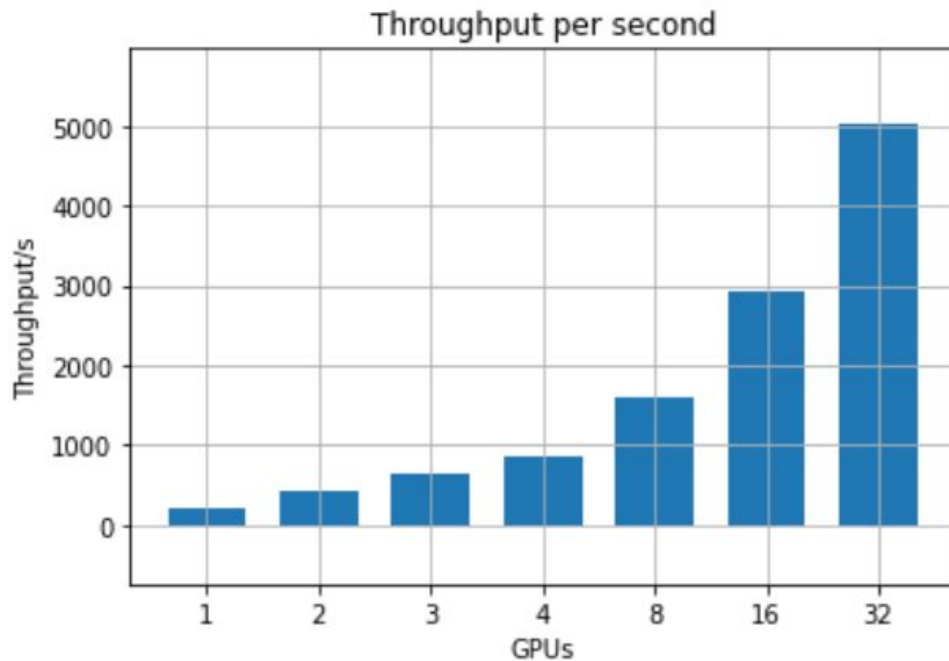
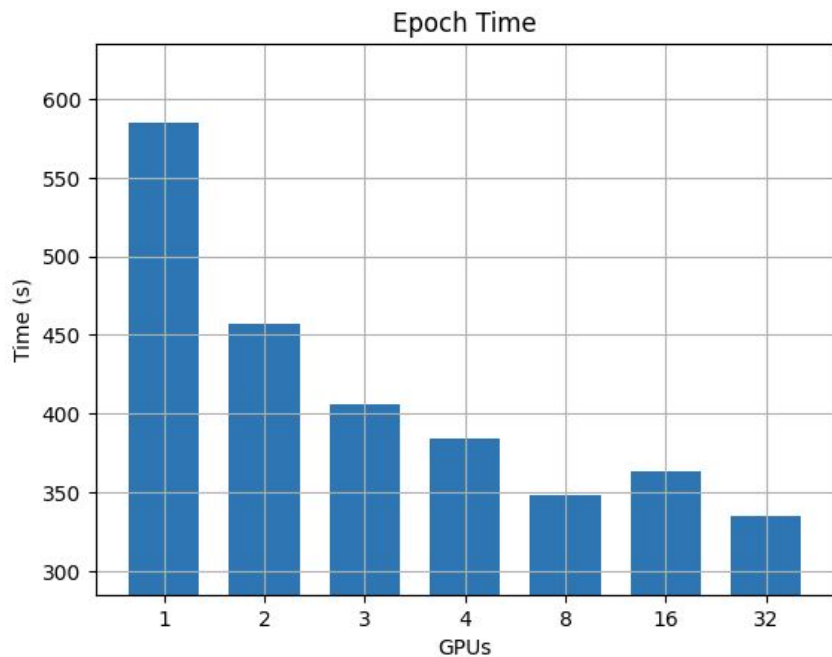
Task 12.1 Reproducing Distributed Training Results on MN5

Results (micro-224):

- Results very similar to the ones reported in the table 12.1.
- Throughput is significantly higher for higher number of GPUs, while the training time slightly decreased up to 40% => very poor scalability.
- Memory usage is not affected by the number of GPUs : tasks are well distributed.

GPUs	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
1	585	216	59
2	457	423	59
3	406	641	59
4	384	844	59
8	348	1591	59
16	363	2913	59
32	335	5008	59

Task 12.1 Reproducing Distributed Training Results on MN5



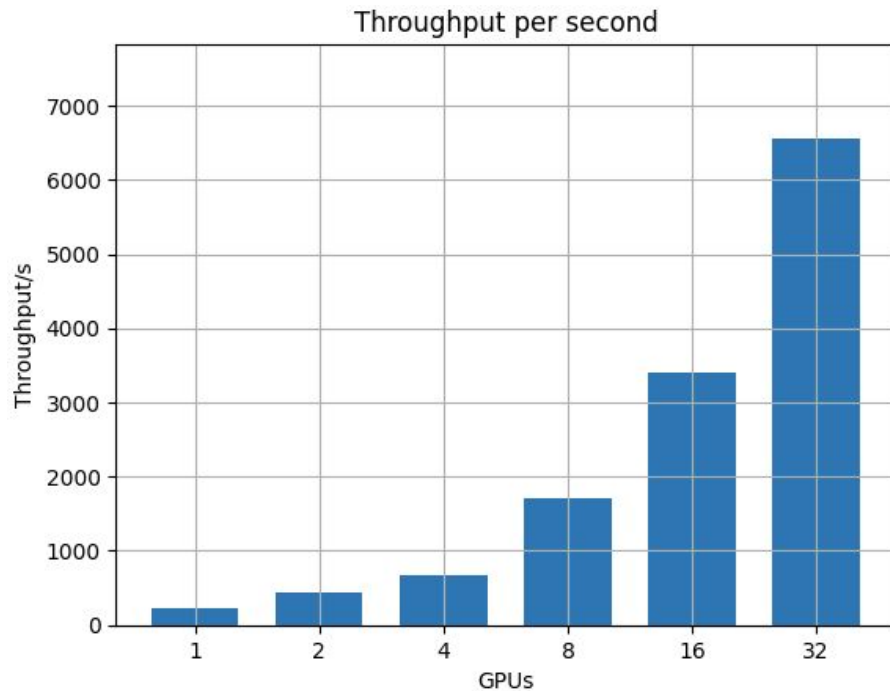
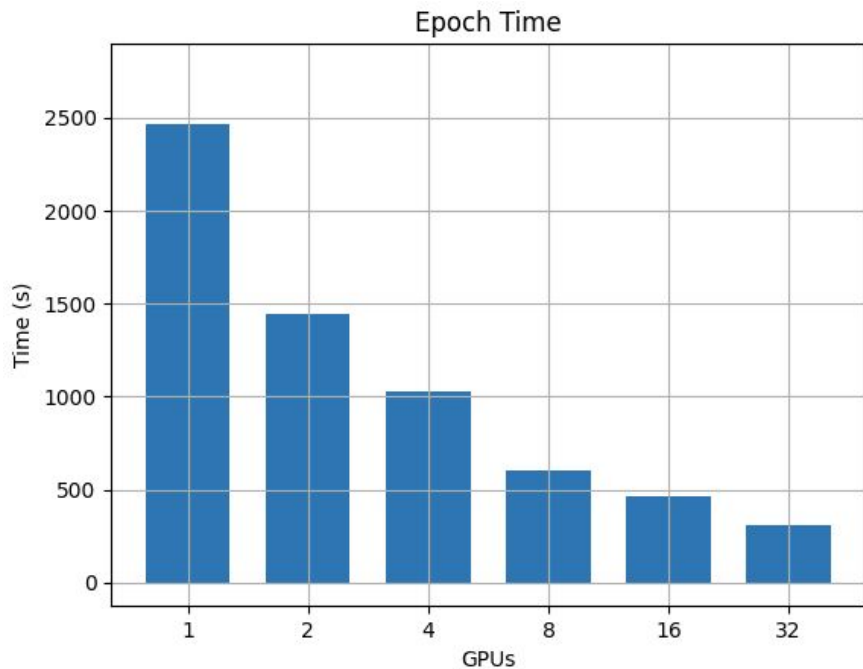
Task 12.2 Analyze and Compare Scaling Efficiency

Results (tiny-224):

- Results very similar to the ones reported in the table 12.5.
- From 1 to 32 GPUs, training time was decreased by 8 : not ideal but very significative.
- Throughput almost doubled at each step. The evolution follows linearly the number of GPUs : efficiency is very high.
- Memory usage is not affected by the number of GPUs : tasks are well distributed.

GPUs	Complete Training Time (s)	Throughput (img/s)	Memory (GB)
1	2465	229	59
2	1446	435	59
4	1027	674	59
8	600	1710	59
16	463	3398	59
32	309	6558	60

Task 12.2 Analyze and Compare Scaling Efficiency



Task 12.2 Analyze and Compare Scaling Efficiency

Analysis (micro-224 VS tiny-224):

- The memory usage is equal regardless of the dataset.
- For low amount of GPUs, `micro-224` does take much less time as the dataset is smaller. However, since the scalability is far superior with `tiny-224`, it eventually fall behind after 32 GPUs.
- This result seems counterintuitive, as the output shows better result the biggest dataset, even overperforming it fully for 32. It seems that the Amdahl's Law does not apply equally.

Task 12.2 Analyze and Compare Scaling Efficiency

- By looking at the number of samples (32 GPUs) :

`micro-224`

```
INFO - ### Total number of parameters 631021000  
INFO - ### Total training samples (batches): 384 (3)
```

`tiny-224`

```
INFO - ### Total number of parameters 631021000  
INFO - ### Total training samples (batches): 3072 (24)
```

- We see that `micro-224` is very small for such number of GPU : only 3 batches. The time taken for training become a minority compared to the set up + other fixed cost. By judging the results, that fixed time could be at around 250/300s. This avoid any linear evolution, at this constant is predominant.
- With `tiny-224`, the model initially takes $8 * 300$, where $7 * 300$ would be due to training. There is way more time to gain, which implies the possibility of a more linear evolution.
- That trend cannot continue though, as the fixed costs remains : we would expect around the same overall time with 64 GPUs.

Task 12.3 Investigate Diminishing Returns in Training Time

- In general, we should expect the completion time to follow such function :

$$t = a + bN + c\#GPUs + d\frac{N}{\#GPUs}$$

with N being the size of the datasets.

- Some costs are fixed and unavoidable (a), some does scale with size but aren't / cannot be parallelized (b), some are only due to the number of GPUs (c) and some are parallelized among GPUs (d).
- If $d \gg a, b, c$, we should expect the model to be highly scalable, as there is much to gain. Otherwise, we are “optimal” in parallelization, which means that no significant progress can be made by raising the number of GPUs anyways.
- If c is big, we can even expect to regress in the long run.

Task 12.3 Investigate Diminishing Returns in Training Time

- Since **tiny-224** manage to recover up to the level of **micro-224**, we can expect the range of costs from **b** to be negligible.
- However as said before, poor scalability does implies big constant overhead (Amdahl's Law consequence).
- Thus, that law is affecting **micro-224** way more, as **a** (and possibly **c**) is predominant over the rest.
- Among the possible factors for overhead, we have :
 - The “startup cost” (**a**)
 - The load of the dataset (**a/c**)
 - The communication overhead (**c**)
 - The validation procedure (**a**)

Task 12.3 Investigate Diminishing Returns in Training Time

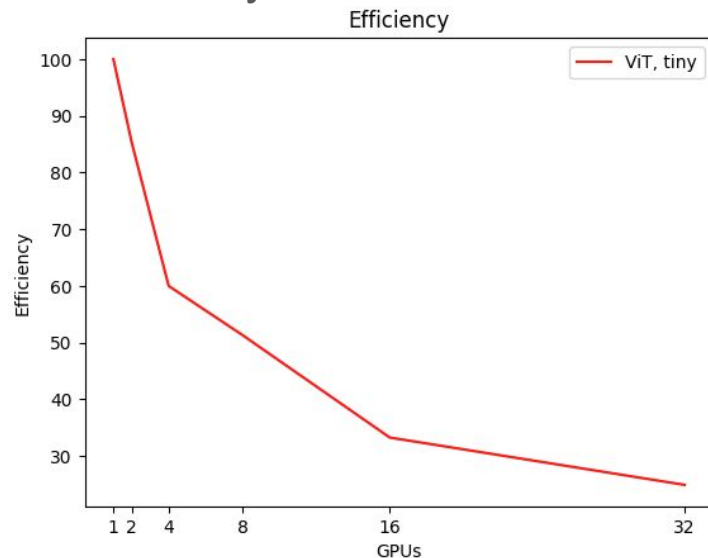
- Consequences : some other areas of improvements must also be considered when building/working with HPC practitioners :
 - Communication speed : better throughput from devices / cables, some architectures can optimise work with certain amount of GPUs. Can also be reducing the number of packets / messages.
 - Loading speed : faster components / better memory.
 - Scripting : more efficient usage of said components, adapting to the practitioner architecture. Maybe maximize the number of parallelized steps is possible.

Task 12.4 Find the Sweet Spot for Your Use Case

- Case a) : You have a tight deadline and unlimited GPU budget
 - One criteria : training time (and possibly throughput).
 - Best configuration is **32** => still improving, is significantly the fastest in time and throughput. Could be with more GPUs, but we don't have enough data to conclude.

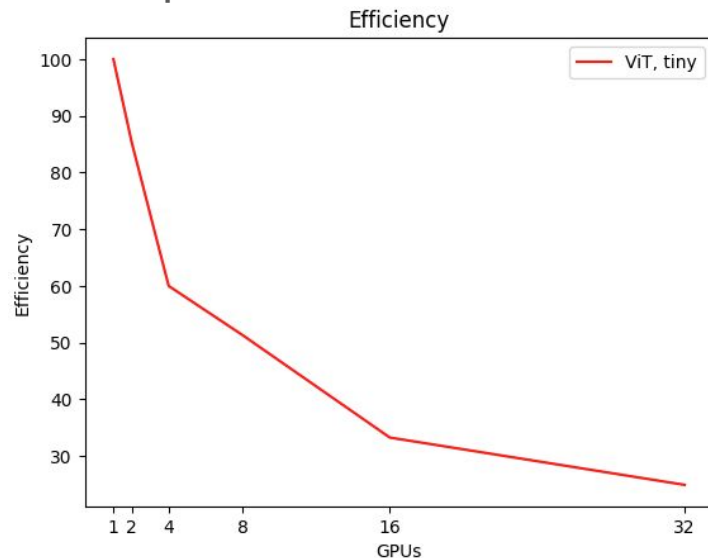
Task 12.4 Find the Sweet Spot for Your Use Case

- Case b) : You need to maximize efficiency under a limited GPU allocation
 - Two criterias : efficiency and low number of GPUs.
 - Best configuration is **1** or **2** => with 4 it drops to 60% and does not improve afterward. Using only one GPU (sequential) is obviously the most “efficient”, but 2 GPUs still have 85% efficiency which is good enough in this case.



Task 12.4 Find the Sweet Spot for Your Use Case

- Case c) : You aim to run as many experiments as possible within a fixed SLURM allocation
 - One criteria : efficiency, since it would result in the best GPU/time ratio.
 - Best configuration is **1** again => it is always more optimal to run **k** experiments of length **t** on **k** GPUs than to run **k/32** experiments of length **t/8** on **k** GPUs, since the time gained won't make up for the raw number of task running simultaneous in the first case.
Parallelization is not meant to achieve this goal.



Task 12.4 Find the Sweet Spot for Your Use Case

- Such considerations are always a balance between resources and time.
- Parallelization is meant to fasten a process at the cost of more hardware utilized. By that mean, it does not provide any advantage if the number of run experiments is the main criteria.
- When using huge supercomputers, it can be assumed that users will always prefer the “brute” approach of maximizing the resources to improve their results (unless we see downgrades).
- However, case a) introduce an “altruistic” point of view. When resources are common with many, it is better for the common good if users are only expanding if they -really- benefit from it, since monopolising GPUs can hamper other with long queues. Concerns from a ecological point could also provide reasons to reserve HPC only when it's necessary.